自动化与机器学习

西湖大学强化学习

课程学习笔记：从 Bellman 方程到 Actor-Critic

Markdown 排印版

学术讲义 • AcSg999

目录

大纲 1

lec1 grid world example 1

Lec2 Bellman equation 4

lec3 Bellman optimality equation 9

lec4 value iteration & policy iteration 14

lec5 Monte carlo learning 18

Lec6 stochastic approximation 22

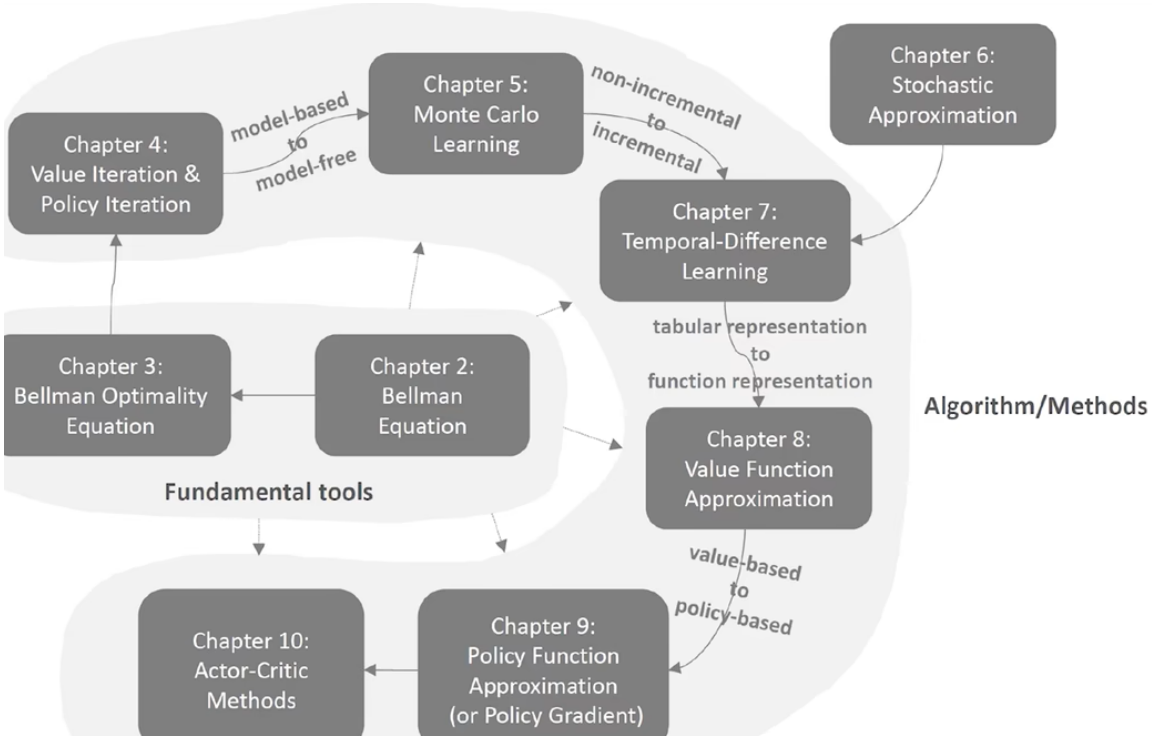
Lec7 temporal difference learning，时序差分 28

lec8 值函数近似 (value function approximation) 35

lec 9 policy gredient （policy function approximation) 39

lec10 actor critic 44

大纲



lec1 grid world example

- state:the status of the agent with respect to the environment
- the set of all state: state space
  action&action space of a state $A(s_i)$
- *Action*: For each state, there are five possible actions: $a_1, \ldots, a_5$
  - a_1: move upwards;
  - a_2: move rightwards;
  - a_3: move downwards;
  - a_4: move leftwards;
  - a_5: stay unchanged;
- state transition : defines the interaction with the environment

probability

- *State transition probability:* use probability to describe state transition!
  - Intuition: At state s_1, if we choose action a_2, the next state is s_2.
  - Math:

$$p(s_2|s_1,a_2)=1$$
$$p(s_i|s_1,a_2)=0 \quad \forall i \neq 2$$

policy tells agents what actions to take at a state, 定义为 π

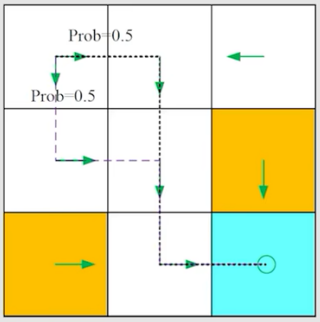
- **Mathematical representation:** using conditional probability

For example, for state s_1:

$$\pi(a_1|s_1)=0$$
$$\pi(a_2|s_1)=1$$
$$\pi(a_3|s_1)=0$$
$$\pi(a_4|s_1)=0$$
$$\pi(a_5|s_1)=0$$

There are stochastic policies.

For example:



In this policy, for s_1:

$$\pi(a_1|s_1)=0$$
$$\pi(a_2|s_1)=0.5$$
$$\pi(a_3|s_1)=0.5$$
$$\pi(a_4|s_1)=0$$
$$\pi(a_5|s_1)=0$$

Reward：a real number we get after an action, 定义为 r

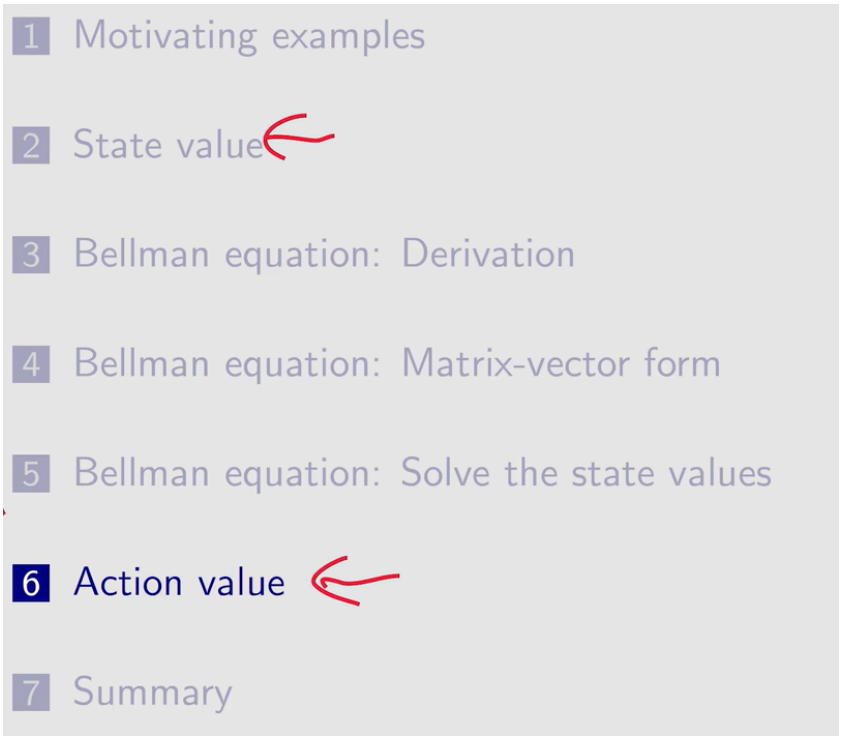
- **Mathematical description**: conditional probability
  - Intuition: At state s_1, if we choose action a_1, the reward is -1.
  - Math: $p(r=-1|s_1,a_1)=1$ and $p(r\neq -1|s_1,a_1)=0$
  - Remarks:
- trajectory is state-action-reward chain
- the return of the trajectory is the sum of reward
- discount rate γ ； discounted return
- an episode is a finite trajectory with a terminal state

MDP: markov decision process

- Key elements of MDP:
  - Sets:
    - State: the set of states $\mathcal{S}$
    - Action: the set of actions $\mathcal{A}(s)$ is associated for state $s\in\mathcal{S}$.
    - Reward: the set of rewards $\mathcal{R}(s,a)$.
  - Probability distribution:
    - State transition probability: at state s, taking action a, the probability to transit to state s' is $p(s'|s,a)$
    - Reward probability: at state s, taking action a, the probability to get reward r is $p(r|s,a)$
  - Policy: at state s, the probability to choose action a is $\pi(a|s)$
  - *Markov property*: memoryless property

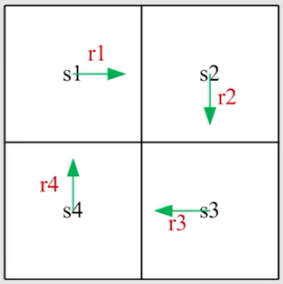
$$p(s_{t+1}|a_{t+1},s_t,\ldots,a_1,s_0)=p(s_{t+1}|a_{t+1},s_t),$$
$$p(r_{t+1}|a_{t+1},s_t,\ldots,a_1,s_0)=p(r_{t+1}|a_{t+1},s_t).$$

Lec2 Bellman equation



Bootstrapping：important idea

- While return is important, how to calculate it?



Method 2:

$$v_1 = r_1 + \gamma(r_2 + \gamma r_3 + \dots) = r_1 + \gamma v_2$$

$$v_2 = r_2 + \gamma(r_3 + \gamma r_4 + \dots) = r_2 + \gamma v_3$$

$$v_3 = r_3 + \gamma(r_4 + \gamma r_1 + \dots) = r_3 + \gamma v_4$$

$$v_4 = r_4 + \gamma(r_1 + \gamma r_2 + \dots) = r_4 + \gamma v_1$$

How to solve these equations? Write in the following matrix-vector form:

$$\underbrace{\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix}}_{\mathbf{v}} = \begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \end{bmatrix} + \begin{bmatrix} \gamma v_2 \\ \gamma v_3 \\ \gamma v_4 \\ \gamma v_1 \end{bmatrix} = \underbrace{\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \end{bmatrix}}_{\mathbf{r}} + \gamma \underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}}_{\mathbf{P}} \underbrace{\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix}}_{\mathbf{v}}$$

which can be rewritten as

$$\mathbf{v} = \mathbf{r} + \gamma \mathbf{P} \mathbf{v}$$

This is the Bellman equation (for this specific deterministic problem)!!

这里 R_{t+1} 是习惯，本质是 R_t ，定义 state value

Consider the following single-step process:

$$S_t \xrightarrow{A_t} R_{t+1}, S_{t+1}$$

- $t, t+1$: discrete time instances
- S_t: state at time t
- A_t: the action taken at state S_t
- R_{t+1}: the reward obtained after taking A_t
- S_{t+1}: the state transited to after taking A_t

Note that S_t, A_t, R_{t+1} are all *random variables*.

Consider the following multi-step trajectory:

$$S_t \xrightarrow{A_t} R_{t+1}, S_{t+1} \xrightarrow{A_{t+1}} R_{t+2}, S_{t+2} \xrightarrow{A_{t+2}} R_{t+3}, \dots$$

The discounted return is

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

- $\gamma \in [0, 1)$ is a discount rate.
- G_t is also a random variable since $R_{t+1}, R_{t+2}, \dots$ are random variables.

The expectation (or called expected value or mean) of G_t is defined as the *state-value function* or simply *state value*:

$$v_\pi(s) = \mathbb{E}[G_t|S_t = s]$$

Remarks:

- It is a function of s. It is a conditional expectation with the condition that the state starts from s.
- It is based on the policy π. For a different policy, the state value may be different.
- It represents the "value" of a state. If the state value is greater, then the policy is better because greater cumulative rewards can be obtained.

补充丨时间下标与价值： S_t, A_t 是第 t 步的状态、动作，R_{t+1} 是执行该动作后得到的奖励；γ 是折扣因子，G_t 是从此刻开始的累计折扣回报。R_t 与 R_{t+1} 不能在同一套时间约定里直接混用。v_π 的下标 π 表示策略；V_k 的下标 k 表示迭代次数。固定策略的价值用于评价它，本身不会自动改变策略。

$$G_t = \sum_{j=0}^{\infty} \gamma^j R_{t+j+1}, \qquad v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s].$$

- 理解：我认为 state value 是基于一个具体的 state 的去评估当前 state 的价值的工具，如果当前 state value 比较大，那么 policy 更加可能选择这个点，当然和 π 和 s 都有关
- from one state , we may have many trajectory, then we have many returns ,and the expectation of all returns is the state value（按轨迹概率加权平均）
  general 公式推导

Second, calculate the second term $\mathbb{E}[G_{t+1}|S_t = s]$:

$$\begin{aligned}
\mathbb{E}[G_{t+1}|S_t = s] &= \sum_{s'} \mathbb{E}[G_{t+1}|S_t = s, S_{t+1} = s']p(s'|s) \\
&= \sum_{s'} \mathbb{E}[G_{t+1}|S_{t+1} = s']p(s'|s) \\
&= \sum_{s'} v_\pi(s')p(s'|s) \\
&= \sum_{s'} v_\pi(s') \sum_a p(s'|s,a)\pi(a|s)
\end{aligned}$$

Therefore, we have

$$
\begin{aligned}
v_\pi(s) &= \mathbb{E}[R_{t+1}|S_t = s] + \gamma \mathbb{E}[G_{t+1}|S_t = s], \\
&= \underbrace{\sum_a \pi(a|s) \sum_r p(r|s,a) r}_{\text{mean of immediate rewards}} + \gamma \underbrace{\sum_a \pi(a|s) \sum_{s'} p(s'|s,a) v_\pi(s')}_{\text{mean of future rewards}}, \\
&= \sum_a \pi(a|s) \left[\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_\pi(s') \right], \quad \forall s \in \mathcal{S}.
\end{aligned}
$$

Highlights:

- The above equation is called the *Bellman equation*, which characterizes the relationship among the state-value functions of different states.

matrix vector formof Bellman equation

Put all these equations for all the states together and rewrite to a matrix-vector form

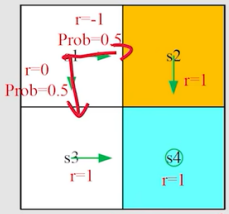
$$v_\pi = r_\pi + \gamma P_\pi v_\pi$$

where

- $v_\pi = [v_\pi(s_1), \ldots, v_\pi(s_n)]^T \in \mathbb{R}^n$
- $r_\pi = [r_\pi(s_1), \ldots, r_\pi(s_n)]^T \in \mathbb{R}^n$
- $P_\pi \in \mathbb{R}^{n \times n}$, where $[P_\pi]_{ij} = p_\pi(s_j|s_i)$, is the *state transition matrix*

If there are four states, $v_\pi = r_\pi + \gamma P_\pi v_\pi$ can be written out as

$$\underbrace{\begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ v_\pi(s_3) \\ v_\pi(s_4) \end{bmatrix}}_{v_\pi} = \underbrace{\begin{bmatrix} r_\pi(s_1) \\ r_\pi(s_2) \\ r_\pi(s_3) \\ r_\pi(s_4) \end{bmatrix}}_{r_\pi} + \gamma \underbrace{\begin{bmatrix} p_\pi(s_1|s_1) & p_\pi(s_2|s_1) & p_\pi(s_3|s_1) & p_\pi(s_4|s_1) \\ p_\pi(s_1|s_2) & p_\pi(s_2|s_2) & p_\pi(s_3|s_2) & p_\pi(s_4|s_2) \\ p_\pi(s_1|s_3) & p_\pi(s_2|s_3) & p_\pi(s_3|s_3) & p_\pi(s_4|s_3) \\ p_\pi(s_1|s_4) & p_\pi(s_2|s_4) & p_\pi(s_3|s_4) & p_\pi(s_4|s_4) \end{bmatrix}}_{P_\pi} \underbrace{\begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ v_\pi(s_3) \\ v_\pi(s_4) \end{bmatrix}}_{v_\pi}$$



For this specific example: 0.5x-1+0.5*

$$\begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ v_\pi(s_3) \\ v_\pi(s_4) \end{bmatrix} = \begin{bmatrix} 0.5(0) + 0.5(-1) \\ 1 \\ 1 \\ 1 \end{bmatrix} + \gamma \begin{bmatrix} 0 & 0.5 & 0.5 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ v_\pi(s_3) \\ v_\pi(s_4) \end{bmatrix}.$$

- Given a policy , finding out the corresponding state values is called policy evaluation.
  实际计算式：这里 V_k 是迭代，不是 entrywise

- Can we avoid the matrix inverse operation? Yes, by iterative algorithms.

  An iterative solution is:

  $$v_{k+1} = r_\pi + \gamma P_\pi v_k$$

  This algorithm leads to a sequence $\{v_0, v_1, v_2, \dots\}$. We can show that

  $$v_k \to v_\pi = (I - \gamma P_\pi)^{-1} r_\pi, \quad k \to \infty$$

action value：the average return the agent can get stating from a state and taking an action, 用 q 表示

Definition:

$$q_\pi(s,a) = \mathbb{E}[G_t|S_t = s, A_t = a]$$

- $q_\pi(s,a)$ is a function of the state-action pair (s,a)
- $q_\pi(s,a)$ depends on π

- It follows from the properties of conditional expectation that

$$\underbrace{\mathbb{E}[G_t|S_t = s]}_{v_\pi(s)} = \sum_a \underbrace{\mathbb{E}[G_t|S_t = s, A_t = a]}_{q_\pi(s,a)} \pi(a|s)$$

Hence,

$$v_\pi(s) = \sum_a \pi(a|s) q_\pi(s,a)$$

lec3 Bellman optimality equation

1. Motivating examples
2. Definition of optimal policy
3. BOE: Introduction
4. BOE: Maximization on the right-hand side
5. BOE: Rewrite as $v = f(v)$
6. Contraction mapping theorem
7. BOE: Solution
8. BOE: Optimality
9. Analyzing optimal policies

optimal policy intro

- The state value could be used to evaluate if a policy is good or not: if

$$v_{\pi_1}(s) \geq v_{\pi_2}(s) \quad \text{for all } s \in \mathcal{S}$$

then π_1 is "better" than π_2.

Definition

A policy π^* is optimal if $v_{\pi^*}(s) \geq v_{\pi}(s)$ for all s and for any other policy π.

$V_{\pi_1}(s)$ 和 $V_{\pi_2}(s)$ 的**物理意义区别不在状态 s 本身，而在"从 s 出发后遵循的策略不同"。**

定义是：

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

意思是：**在状态 s 下，如果之后一直按照策略 π 行动，智能体能获得的期望累计回报。**

所以：

- $V_{\pi_1}(s)$：从状态 s 出发，之后按策略 π_1 行动，平均能拿多少回报。
- $V_{\pi_2}(s)$：从同一个状态 s 出发，之后按策略 π_2 行动，平均能拿多少回报。

贝尔曼最优公式 BOE，箭头为已知量，$V(s)$ 代求量

- **Bellman optimality equation (elementwise form):**

$$v(s) = \max_\pi \sum_a \pi(a|s) \left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v(s') \right), \quad \forall s \in \mathcal{S}$$
$$= \max_\pi \sum_a \pi(a|s) q(s,a) \quad s \in \mathcal{S}$$

Bellman optimality equation (matrix-vector form):

$$v = \max_{\pi}(r_\pi + \gamma P_\pi v)$$

BOE is tricky yet elegant!

- Why elegant? It describes the optimal policy and optimal state value in an elegant way.
- Why tricky? There is a maximization on the right-hand side, which may not be straightforward to see how to compute.

如何求解：这里的 q_1, q_2, q_3 指的是所有不同 action（a）对应的 $q(s, a_1), \ldots$，Q 是 action value function/Q function

$$= \max_{\pi} \sum_{a} \pi(a|s) q(s, a)$$

Example (How to solve $\max_\pi \sum_a \pi(a|s)q(s,a)$)

Suppose $q_1, q_2, q_3 \in \mathbb{R}$ are given. Find c_1^*, c_2^*, c_3^* solving

$$\max_{c_1, c_2, c_3} c_1 q_1 + c_2 q_2 + c_3 q_3.$$

where $c_1 + c_2 + c_3 = 1$ and $c_1, c_2, c_3 \geq 0$.

Without loss of generality, suppose $q_3 \geq q_1, q_2$. Then, the optimal solution is $c_3^* = 1$ and $c_1^* = c_2^* = 0$. That is because for any c_1, c_2, c_3

$$q_3 = (c_1 + c_2 + c_3)q_3 = c_1 q_3 + c_2 q_3 + c_3 q_3 \geq c_1 q_1 + c_2 q_2 + c_3 q_3.$$

所以可以处理这个最优项 $\max_\pi$

Inspired by the above example, considering that $\sum_a \pi(a|s) = 1$, we have

$$\max_{\pi} \sum_{a} \pi(a|s) q(s, a) = \max_{a \in \mathcal{A}(s)} q(s, a),$$

where the optimality is achieved when

$$\pi(a|s) = \begin{cases} 1 & a = a^* \\ 0 & a \neq a^* \end{cases}$$

prelimiaries: fix point and contraction mapping

Theorem (Contraction Mapping Theorem)

For any equation that has the form of $x = f(x)$, if f is a contraction mapping, then

- *Existence: there exists a fixed point x^* satisfying $f(x^*) = x^*$.*
- *Uniqueness: The fixed point x^* is unique.*
- *Algorithm: Consider a sequence $\{x_k\}$ where $x_{k+1} = f(x_k)$, then $x_k \to x^*$ as $k \to \infty$. Moreover, the convergence rate is exponentially fast.*

Let's come back to the Bellman optimality equation:

$$v = f(v) = \max_{\pi}(r_\pi + \gamma P_\pi v)$$

Theorem (Contraction Property)

$f(v)$ is a contraction mapping satisfying

$$\|f(v_1) - f(v_2)\| \leq \gamma \|v_1 - v_2\|$$

where γ is the discount rate!

BOE 是一个特殊的贝尔曼公式

- What does an optimal policy π^* look like?

Theorem (Greedy Optimal Policy)

For any $s \in \mathcal{S}$, the deterministic greedy policy

$$\pi^*(a|s) = \begin{cases} 1 & a = a^*(s) \\ 0 & a \neq a^*(s) \end{cases}$$

is an optimal policy solving the BOE. Here,

$$a^*(s) = \arg\max_a q^*(a, s),$$

where $q^*(s,a) := \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v^*(s')$.

已知红色的量求黑色的量

- It can be clearly seen from the BOE

$$v(s) = \max_\pi \sum_a \pi(a|s) \left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v(s') \right)$$

  that there are three factors:
  - Reward design: r
  - System model: $p(s'|s,a)$, $p(r|s,a)$
  - Discount rate: γ
  - $v(s), v(s'), \pi(a|s)$ are unknowns to be calculated

- if r becomes $ar + b$

lec4 value iteration & policy iteration

value iteration, 即 BOE 的解法中的值迭代

- ▷ How to solve the Bellman optimality equation?

$$v = f(v) = \max_{\pi}(r_\pi + \gamma P_\pi v)$$

▷ In the last lecture, we know that the contraction mapping theorem suggest an iterative algorithm:

$$v_{k+1} = f(v_k) = \max_{\pi}(r_\pi + \gamma P_\pi v_k), \quad k = 1, 2, 3 \ldots$$

where v_0 can be arbitrary.

▷ This algorithm can eventually find the optimal state value and an optimal policy.

▷ This algorithm is called *value iteration*!

- it can be decomposed into two steps.1. 给定 V_k 求一个最优的 π_{k+1} 2. 用这个策略去更新 V
- 停止条件：$V_{k+1} - V_k$ 接近于 0
- v_k is not state value , just a iterating value
  policy iteration 初始化一个 π_k ， 得到 V_{π_k} 然后用它求 π_{k+1}

- ▷ Algorithm description:

Given a random initial policy π_0,

- Step 1: policy evaluation (PE)

  This step is to calculate the state value of π_k

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

  Note that v_{π_k} is a state value function.

- Step 2: policy improvement (PI)

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k})$$

  The maximization is componentwise!

四个问题

- Q1: In the policy evaluation step, how to get the state value v_{π_k} by solving the Bellman equation?
- Q2: In the policy improvement step, why is the new policy π_{k+1} better than π_k?
- Q3: Why such an iterative algorithm can finally reach an optimal policy?
- Q4: What is the relationship between this policy iteration algorithm and the previous value iteration algorithm?

1. 在每一步的 policy evaluation 中还有个小迭代（小循环）

- Iterative solution:

$$v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \ldots$$

▷ Already studied in the lecture on Bellman equation.

▷ Policy iteration is an iterative algorithm with another iterative algorithm embedded in the policy evaluation step!

- 2.lemma: 如果 π_{k+1} 是这样算出来的，那么 $V_{\pi_{k+1}} > V_{\pi_k}$

补充丨策略改进的边界：这里一般保证的是逐状态“不变差”，即 $V_{\pi_{k+1}}(s) \geq V_{\pi_k}(s)$；不保证每个状态都严格变好，达到最优时可以相等。

伪代码

While the policy has not converged, for the kth iteration, do

Policy evaluation:

Initialization: an arbitrary initial guess $v_{\pi_k}^{(0)}$

While $v_{\pi_k}^{(j)}$ has not converged, for the jth iteration, do

For every state $s \in \mathcal{S}$, do

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}^{(j)}(s') \right]$$

Policy improvement:

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

$$q_{\pi_k}(s,a) = \sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s')$$

$$a_k^*(s) = \arg\max_a q_{\pi_k}(s,a)$$

$$\pi_{k+1}(a|s) = 1 \text{ if } a = a_k^*, \text{ and } \pi_{k+1}(a|s) = 0 \text{ otherwise}$$

迭代过程

- Setting: $r_{\text{boundary}} = -1$, $r_{\text{forbidden}} = -10$, $r_{\text{target}} = 1$, $\gamma = 0.9$.

> 迭代早期中间表格上黄色格子的状态价值 $v(yellow) = 10$。
>
> 旁边的普通状态s，动作向右，走入黄色 s'_{yellow}：

$$Q(s, a_{\text{right}}) = r + \gamma\, v(s'_{yellow})$$

$$Q = -10 + 0.9 \times 10 = -1$$

算出来 Q=-1。

旁边别的动作（比如向上）普通走一步：每走一步基础代价也是 $r = -1$。

- truncated policy iteration

先对比 value iteration and policy iteration，这里蓝色的关键不同是 v_0, v_1 等等只是中间的迭代向量，不当成精确价值函数，而 policy 这里是算出在新策略下对应的帧数价值函数

-

	Policy iteration algorithm	Value iteration algorithm	Comments
1) Policy:	π_0	N/A	
2) Value:	$v_{\pi_0} = r_{\pi_0} + \gamma P_{\pi_0} v_{\pi_0}$	$v_0 := v_{\pi_0}$	
3) Policy:	$\pi_1 = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_0})$	$\pi_1 = \arg\max_\pi (r_\pi + \gamma P_\pi v_0)$	The two policies are the same
4) Value:	$v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$	$v_1 = r_{\pi_1} + \gamma P_{\pi_1} v_0$	$v_{\pi_1} \geq v_1$ since $v_{\pi_1} \geq v_{\pi_0}$
5) Policy:	$\pi_2 = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_1})$	$\pi_2' = \arg\max_\pi (r_\pi + \gamma P_\pi v_1)$	
⋮	⋮	⋮	⋮

截断的理解，实际当中不计算无穷步，truncated policy 把后面到无限的迭代给截断

-

Consider the step of solving $v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$:

$$v_{\pi_1}^{(0)} = v_0$$

value iteration $\leftarrow v_1 \leftarrow$ $v_{\pi_1}^{(1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)}$

$$v_{\pi_1}^{(2)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)}$$

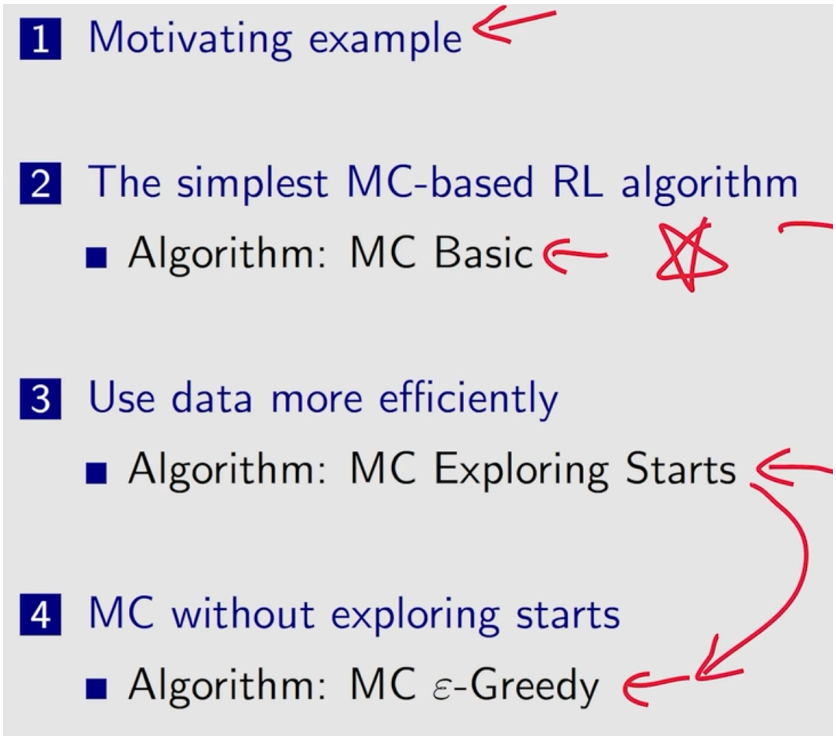
$$\vdots$$

truncated policy iteration $\leftarrow \bar{v}_1 \leftarrow$ $v_{\pi_1}^{(j)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)}$

$$\vdots$$

policy iteration $\leftarrow v_{\pi_1} \leftarrow$ $v_{\pi_1}^{(\infty)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)}$

lec5 Monte carlo learning



- MBRL=model baesd reinforcement learning=value iteration+policy iteration
  law of large numbers 大数定律，大量采样平均值等于期望值
- For a random variable X. Suppose $\{x_j\}_{j=1}^N$ are some iid samples. Let $\bar{x} = \frac{1}{N}\sum_{j=1}^N x_j$ be the average of the samples. Then,

$$\mathbb{E}[\bar{x}] = \mathbb{E}[X],$$
$$\mathrm{Var}[\bar{x}] = \frac{1}{N}\mathrm{Var}[X].$$

  As a result, $\bar{x}$ is an unbiased estimate of $\mathbb{E}[X]$ and its variance decreases
- MC basic：use policy iteration but model free

policy iteration–>policy improment–> 这一步要求不同的 action value 并取最大的一个–>action value 的计算方法有 model based（已知转移概率 $P(s' \mid s, a)$，奖励函数等等）和根本定义（model free 采样）G_t 是 return 从时刻 t 开始的完整回报

- **Expression 1 requires the model:**

$$q_{\pi_k}(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v_{\pi_k}(s')$$

- **Expression 2 does not require the model:**

$$q_{\pi_k}(s,a) = \mathbb{E}[G_t|S_t = s, A_t = a]$$

The procedure of Monte Carlo estimation of action values:

- Starting from (s,a), following policy π_k, generate an episode.
- The return of this episode is $g(s,a)$
- $g(s,a)$ is a sample of G_t in

$$q_{\pi_k}(s,a) = \mathbb{E}[G_t|S_t = s, A_t = a]$$

- Suppose we have a set of episodes and hence $\{g^{(j)}(s,a)\}$. Then,

$$q_{\pi_k}(s,a) = \mathbb{E}[G_t|S_t = s, A_t = a] \approx \frac{1}{N}\sum_{i=1}^{N} g^{(i)}(s,a).$$

如何理解这里的随机性和采样：reward 随机，状态转移随机，策略 π_k 确定，执行 action a 后进入的 s' 不确定。data=sample=experience

Pseudocode: MC Basic algorithm (a model-free variant of policy iteration)

Initialization: Initial guess π_0.

Aim: Search for an optimal policy.

While the value estimate has not converged, for the kth iteration, do

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

Collect sufficiently many episodes starting from (s, a) following π_k

MC-based policy evaluation step:

$q_{\pi_k}(s, a) =$ average return of all the episodes starting from (s, a)

Policy improvement step:

$a_k^*(s) = \arg\max_a q_{\pi_k}(s, a)$

$\pi_{k+1}(a|s) = 1$ if $a = a_k^*$, and $\pi_{k+1}(a|s) = 0$ otherwise

- eposide length，探索长度，action value 为正数代表最终会达到目标

Estimated state value and policy with episode length=3

Estimated state value and policy with episode length=4

- 5.2 improving the MC basic, exploring starts
  - visit, 把一个 (s, a) pair 作为一次 visit
  - initial-visit method,first-visit method,every-visit method
  - 在**一条完整 episode 里**，只有**第一次**遇到 (s, a) 的时候，才把这条轨迹从这个点往后得到的回报 G 拿来更新、估计 $q_\pi(s, a)$；在**一条完整 episode 里**，**每一次**遇到 (s, a)，都把从该点往后得到的回报 G 拿来更新 $q_\pi(s, a)$。
- further more, exploring starts 需要从每个 s 和 a 出发保证每个 visit 都有，所以实际很难使用
- 我们用 epsilon greedy 算法改进

soft：a policy is soft if and only if the probability to take any action is positive，这样不需要从每个 s, a 出发也能探索完全部

▷ What soft policies will we use? Answer: ε-greedy policies

- **What is an ε-greedy policy?**

$$\pi(a|s) = \begin{cases} 1 - \dfrac{\varepsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)| - 1), & \text{for the greedy action,} \\ \dfrac{\varepsilon}{|\mathcal{A}(s)|}, & \text{for the other } |\mathcal{A}(s)| - 1 \text{ actions.} \end{cases}$$

where $\varepsilon \in [0, 1]$ and $|\mathcal{A}(s)|$ is the number of actions for s.

– greedy 指的是 action value 最大的那个，设计成这样使得选择 greedy action 概率始终高于其他 action

– exploitation 利用 exploration 探索
区别于之前的一般贪心策略，直接选择最大的 q_{π_k}

▷ **How to embed ε-greedy into the MC-based RL algorithms?**

Now, the policy improvement step is changed to solve

$$\pi_{k+1}(s) = \arg\max_{\pi \in \Pi_\varepsilon} \sum_a \pi(a|s) q_{\pi_k}(s, a),$$

where Π_ε denotes the set of all ε-greedy policies with a fixed value of ε.

The optimal policy here is

$$\pi_{k+1}(a|s) = \begin{cases} 1 - \dfrac{|\mathcal{A}(s)|-1}{|\mathcal{A}(s)|}\varepsilon, & a = a_k^*, \\ \dfrac{1}{|\mathcal{A}(s)|}\varepsilon, & a \neq a_k^*. \end{cases}$$

For each episode, do

Episode generation: Randomly select a starting state-action pair (s_0, a_0). Following the current policy, generate an episode of length T: $s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T$.

Policy evaluation and policy improvement:

Initialization: $g \leftarrow 0$

For each step of the episode, $t = T-1, T-2, \ldots, 0$, do

$g \leftarrow \gamma g + r_{t+1}$

Use the every-visit method:

If (s_t, a_t) does not appear in $(s_0, a_0, s_1, a_1, \ldots, s_{t-1}, a_{t-1})$, then

$Returns(s_t, a_t) \leftarrow Returns(s_t, a_t) + g$

$q(s_t, a_t) = \text{average}(Returns(s_t, a_t))$

Let $a^* = \arg\max_a q(s_t, a)$ and

$$\pi(a|s_t) = \begin{cases} 1 - \dfrac{|\mathcal{A}(s_t)|-1}{|\mathcal{A}(s_t)|}\epsilon, & a = a^* \\ \dfrac{1}{|\mathcal{A}(s_t)|}\epsilon, & a \neq a^* \end{cases}$$

example of epislon greedy, ϵ 不能太大，或者一开始较大后面逐渐减小比较好

–

Lec6 stochastic approximation

motivating example: mean estimation, 求和与迭代式

- Therefore, we obtain the following iterative algorithm:

$$w_{k+1} = w_k - \frac{1}{k}(w_k - x_k).$$

- W_{k+1} 是前 k 个的平均
  Robbins-Monro algorithm 目标是求解 $g(w) = 0$ 的 w，不就是梯度下降吗

- The Robbins-Monro (RM) algorithm can solve this problem:

$$w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k), \qquad k = 1, 2, 3, \ldots$$

where

- w_k is the kth estimate of the root
- $\tilde{g}(w_k, \eta_k) = g(w_k) + \eta_k$ is the kth noisy observation
- a_k is a positive coefficient.

an example，在带噪声的观测下找根

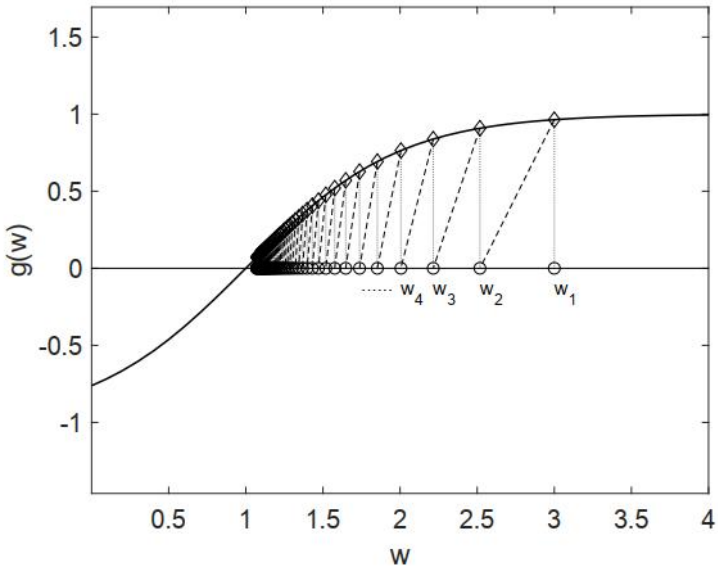


Figure 6.2: An example to illustrate the convergence of the RM algorithm.

① 当前估计值 w_k

例如：猜测根的位置

↓

② 获取带噪观测值

$$\tilde{g}(w_k) = w_k^3 - 5 + \eta_k$$

↓

③ 按照观测结果修正估计

$$w_{k+1} = w_k - \frac{1}{k}\tilde{g}(w_k)$$

↓

④ 重复迭代

希望估计值逐渐接近真实根 1.71

解读 theorem 1. 要求梯度为正数且有上界，即函数递增 2. a_k converge to 0 as k 趋近于无穷，且 a_k dont converge to 0 too fast, $a_k = 1/k$ 是满足的一种情况

- **Theorem (Robbins-Monro Theorem)**

  In the Robbins-Monro algorithm, if

  1) $0 < c_1 \leq \nabla_w g(w) \leq c_2$ *for all* w;

  2) $\sum_{k=1}^{\infty} a_k = \infty$ *and* $\sum_{k=1}^{\infty} a_k^2 < \infty$;

  3) $\mathbb{E}[\eta_k | \mathcal{H}_k] = 0$ *and* $\mathbb{E}[\eta_k^2 | \mathcal{H}_k] < \infty$;

  where $\mathcal{H}_k = \{w_k, w_{k-1}, \dots\}$, *then* w_k *converges with probability 1 (w.p.1) to the root* w^* *satisfying* $g(w^*) = 0$.

用 RM 算法升级之前的 mean estimation

- 1) Consider a function:

$$g(w) \doteq w - \mathbb{E}[X].$$

Our aim is to solve $g(w) = 0$. If we can do that, then we can obtain $\mathbb{E}[X]$.

2) The observation we can get is

$$\tilde{g}(w, x) \doteq w - x,$$

because we can only obtain samples of X. Note that

$$\begin{aligned}\tilde{g}(w, \eta) &= w - x = w - x + \mathbb{E}[X] - \mathbb{E}[X] \\ &= (w - \mathbb{E}[X]) + (\mathbb{E}[X] - x) \doteq g(w) + \eta,\end{aligned}$$

3) The RM algorithm for solving $g(x) = 0$ is

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k (w_k - x_k),$$

拓展, w.p. 1 =with possibility 1

Theorem (Dvoretzky's Theorem)

Consider a stochastic process

$$w_{k+1} = (1 - \alpha_k)w_k + \beta_k \eta_k,$$

- *where* $\{\alpha_k\}_{k=1}^{\infty}, \{\beta_k\}_{k=1}^{\infty}, \{\eta_k\}_{k=1}^{\infty}$ *are stochastic sequences. Here* $\alpha_k \geq 0, \beta_k \geq 0$ *for all* k*. Then,* w_k *would converge to zero with probability 1 if the following conditions are satisfied:*

  1) $\sum_{k=1}^{\infty} \alpha_k = \infty$, $\sum_{k=1}^{\infty} \alpha_k^2 < \infty$; $\sum_{k=1}^{\infty} \beta_k^2 < \infty$ *uniformly w.p.1;*

  2) $\mathbb{E}[\eta_k | \mathcal{H}_k] = 0$ *and* $\mathbb{E}[\eta_k^2 | \mathcal{H}_k] \leq C$ *w.p.1;*

where $\mathcal{H}_k = \{w_k, w_{k-1}, \ldots, \eta_{k-1}, \ldots, \alpha_{k-1}, \ldots, \beta_{k-1}, \ldots\}$.

SGD stochastic gradient descent ——and BGD, MBGD

Suppose we aim to solve the following optimization problem:

$$\min_w \quad J(w) = \mathbb{E}[f(w, X)]$$

- w is the parameter to be optimized.
- X is a random variable. The expectation is with respect to X.
- w and X can be either scalars or vectors. The function $f(\cdot)$ is a scalar.

Method 3: stochastic gradient descent (SGD)

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k),$$

- Compared to the gradient descent method: Replace the true gradient $\mathbb{E}[\nabla_w f(w_k, X)]$ by the stochastic gradient $\nabla_w f(w_k, x_k)$.
- Compared to the batch gradient descent method: let $n = 1$.

- The GD algorithm for solving the above problem is

$$\begin{aligned} w_{k+1} &= w_k - \alpha_k \nabla_w J(w_k) \\ &= w_k - \alpha_k \mathbb{E}[\nabla_w f(w_k, X)] \\ &= w_k - \alpha_k \mathbb{E}[w_k - X]. \end{aligned}$$

- The SGD algorithm for solving the above problem is

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k) = w_k - \alpha_k (w_k - x_k)$$

- 相当于把随机变量 X 随机采一个样来代替期望，the mean estimation is a special case of SGD
证明 SGD 是一个特殊的 RM 算法：把求解最优的问题转化为求解梯度为 0 的问题

- The aim of SGD is to minimize

$$J(w) = \mathbb{E}[f(w, X)]$$

This problem can be converted to a root-finding problem:

$$\nabla_w J(w) = \mathbb{E}[\nabla_w f(w, X)] = 0$$

Let

$$g(w) = \nabla_w J(w) = \mathbb{E}[\nabla_w f(w, X)].$$

Then, the aim of SGD is to find the root of $g(w) = 0$.

$\tilde{g}$ 是有噪音的测量，可以分成 $g+\eta$，所以这个情况下的 RM 就是 SGD

- What we can measure is

$$\tilde{g}(w,\eta) = \nabla_w f(w,x)$$
$$= \underbrace{\mathbb{E}[\nabla_w f(w,X)]}_{g(w)} + \underbrace{\nabla_w f(w,x) - \mathbb{E}[\nabla_w f(w,X)]}_{\eta}.$$

Then, the RM algorithm for solving $g(w)=0$ is

$$w_{k+1} = w_k - a_k \tilde{g}(w_k,\eta_k) = w_k - a_k \nabla_w f(w_k,x_k). \leftarrow SGD$$

- It is exactly the SGD algorithm.
- Therefore, SGD is a special RM algorithm.

研究 SGD 收敛的 pattern 这里是二阶梯度不是平方，δ 是 error，利用凸函数二阶导恒大于零性质

- To answer this question, we consider the relative error between the stochastic and batch gradients:

$$\delta_k \doteq \frac{|\nabla_w f(w_k,x_k) - \mathbb{E}[\nabla_w f(w_k,X)]|}{|\mathbb{E}[\nabla_w f(w_k,X)]|}.$$

Since $\mathbb{E}[\nabla_w f(w^*,X)] = 0$, we further have

$$\delta_k = \frac{|\nabla_w f(w_k,x_k) - \mathbb{E}[\nabla_w f(w_k,X)]|}{|\mathbb{E}[\nabla_w f(w_k,X)] - \mathbb{E}[\nabla_w f(w^*,X)]|} = \frac{|\nabla_w f(w_k,x_k) - \mathbb{E}[\nabla_w f(w_k,X)]|}{|\mathbb{E}[\nabla_w^2 f(\tilde{w}_k,X)(w_k-w^*)]|}.$$

where the last equality is due to the mean value theorem and $\tilde{w}_k \in [w_k, w^*]$.

$f(x_1) - f(x_2) = f'(x_3)(x_1 - x_2)$

$$\delta_k \leq \frac{|\overbrace{\nabla_w f(w_k,x_k)}^{\text{stochastic gradient}} - \overbrace{\mathbb{E}[\nabla_w f(w_k,X)]}^{\text{true gradient}}|}{\underbrace{c|w_k - w^*|}_{\text{distance to the optimal solution}}}.$$

和深度学习中学的 SGD 中 x 的概念的不同的厘清：深度学习中没有随机变量，可以引入随机变量

In particular, suppose X is a random variable defined on the set $\{x_i\}_{i=1}^n$. Suppose its probability distribution is uniform such that

$$p(X = x_i) = 1/n$$

Then, the deterministic optimization problem becomes a stochastic one:

$$\min_w \quad J(w) = \frac{1}{n}\sum_{i=1}^{n} f(w, x_i) = \mathbb{E}[f(w, X)].$$

SGD，BGD，MBGD 关系，MBGD 抽取 m 个是有放回抽样！

- If $m = 1$, MBGD becomes SGD.
- If $m = n$, MBGD does NOT become BGD strictly speaking because MBGD uses randomly fetched n samples whereas BGD uses all n numbers. In particular, MBGD may use a value in $\{x_i\}_{i=1}^n$ multiple times whereas BGD uses each number once.

Lec7 temporal difference learning，时序差分

Third, consider an even more complex problem: calculate

$$w = \mathbb{E}[R + \gamma v(X)],$$

where R, X are random variables, γ is a constant, and $v(\cdot)$ is a function.

- Suppose we can obtain samples $\{x\}$ and $\{r\}$ of X and R. we define

$$\begin{aligned} g(w) &= w - \mathbb{E}[R + \gamma v(X)], \\ \tilde{g}(w, \eta) &= w - [r + \gamma v(x)] \\ &= (w - \mathbb{E}[R + \gamma v(X)]) + (\mathbb{E}[R + \gamma v(X)] - [r + \gamma v(x)]) \\ &\doteq g(w) + \eta. \end{aligned}$$

- Then, the problem becomes a root-finding problem: $g(w) = 0$. The corresponding RM algorithm is

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k[w_k - (r_k + \gamma v(x_k))]$$

- 同时采样 X 和 R

$V(s)$ -> 近似 s 的 state value，$V_t(S_t)$ ，$\bar{V}_t$,the algorithm drives $v(s_t)$ to $\bar{V}_t$

- **The TD learning algorithm is**

$$\begin{cases} v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\big[v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]\big], & (1) \\ v_{t+1}(s) = v_t(s), \quad \forall s \neq s_t, & (2) \end{cases}$$

The TD algorithm can be annotated as

$$\underbrace{v_{t+1}(s_t)}_{\text{new estimate}} = \underbrace{v_t(s_t)}_{\text{current estimate}} - \alpha_t(s_t)\Big[\underbrace{v_t(s_t) - \underbrace{[r_{t+1} + \gamma v_t(s_{t+1})]}_{\text{TD target } \bar{v}_t}}_{\text{TD error } \delta_t}\Big], \quad (3)$$

TD error: 这里包含了两个时刻，表明现在的 V 和实际的 state value 的差异

- It reflects the deficiency between v_t and v_π. To see that, denote

$$\delta_{\pi,t} \doteq v_\pi(s_t) - [r_{t+1} + \gamma v_\pi(s_{t+1})]$$

Note that

$$\mathbb{E}[\delta_{\pi,t}|S_t = s_t] = v_\pi(s_t) - \mathbb{E}\big[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s_t\big] = 0.$$

- If $v_t = v_\pi$, then δ_t should be zero (in the expectation sense).
- Hence, if δ_t is not zero, then v_t is not equal to v_π.

TD 算法是用来在 model free 的情况下解贝尔曼最优公式 of a given policy，下面是贝尔曼期望 equation

$$v_\pi(s) = \mathbb{E}\big[R + \gamma v_\pi(S')|S = s\big], \quad s \in \mathcal{S}. \quad (5)$$

Equation (5) is another expression of the Bellman equation. It is

Theorem (Convergence of TD Learning)

By the TD algorithm (1), $v_t(s)$ *converges with probability 1 to* $v_\pi(s)$ *for all* $s \in \mathcal{S}$ *as* $t \to \infty$ *if* $\sum_t \alpha_t(s) = \infty$ *and* $\sum_t \alpha_t^2(s) < \infty$ *for all* $s \in \mathcal{S}$.

Remarks:

- This theorem says the state value can be found by the TD algorithm for a given a policy π.
- $\sum_t \alpha_t(s) = \infty$ and $\sum_t \alpha_t^2(s) < \infty$ must be valid for all $s \in \mathcal{S}$. At time step t, if $s = s_t$ which means that s is visited at time t, then $\alpha_t(s) > 0$; otherwise, $\alpha_t(s) = 0$ for all the other $s \neq s_t$. That requires every state must be visited an infinite (or sufficiently many) number of times.
- The learning rate α is often selected as a small constant. In this case, the condition that $\sum_t \alpha_t^2(s) < \infty$ is invalid anymore. When α is constant, it can still be shown that the algorithm converges in the sense of expectation sense.

MC 需要算出 G_t，所以会一直算下去，TD 用了采样去更新

TD/Sarsa learning	MC learning
Online: TD learning is online. It can update the state/action values immediately after receiving a reward.	**Offline:** MC learning is offline. It has to wait until an episode has been completely collected.
Continuing tasks: Since TD learning is online, it can handle both episodic and continuing tasks.	**Episodic tasks:** Since MC learning is offline, it can only handle episodic tasks that has terminate states.

In particular, by defining

$$g(v(s)) = v(s) - \mathbb{E}\big[R + \gamma v_\pi(S') | s\big],$$

we can rewrite (5) as

$$g(v(s)) = 0.$$

Therefore, the RM algorithm for solving $g(v(s)) = 0$ is

$$\begin{aligned} v_{k+1}(s) &= v_k(s) - \alpha_k \tilde{g}(v_k(s)) \\ &= v_k(s) - \alpha_k \Big(v_k(s) - \big[r_k + \gamma v_\pi(s'_k)\big]\Big), \quad k = 1, 2, 3, \ldots \end{aligned}$$

variance 怎么理解：方差，即依赖随机变量的多少

TD/Sarsa learning	MC learning
Bootstrapping: TD bootstraps because the update of a value relies on the previous estimate of this value. Hence, it requires initial guesses.	**Non-bootstrapping:** MC is not bootstrapping, because it can directly estimate state/action values without any initial guess.
Low estimation variance: TD has lower than MC because there are fewer random variables. For instance, Sarsa requires $R_{t+1}, S_{t+1}, A_{t+1}$.	**High estimation variance:** To estimate $q_\pi(s_t, a_t)$, we need samples of $R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$. Suppose the length of each episode is L. There are $\lvert\mathcal{A}\rvert^L$ possible episodes.

-
- sarsa

  directly estimate action values，q_t 是对 q_π （action）的估计

  - We can use the following *Sarsa* algorithm to estimate the action values:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})]\right],$$
$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t),$$

  $(S_t, a_t, r_{t+1}, S_{t+1}, a_{t+1})$ sarsa+policy improvement（update）即用这个 sarsa 做 policy evaluation

  - For each episode, do
    - If the current s_t is not the target state, do
      - Collect the experience $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$: In particular, take action a_t following $\pi_t(s_t)$, generate r_{t+1}, s_{t+1} and then take action a_{t+1} following $\pi_t(s_{t+1})$.
      - *Update q-value:*

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})]\right]$$

      - *Update policy:*

$$\pi_{t+1}(a|s_t) = 1 - \frac{\epsilon}{|\mathcal{A}|}(|\mathcal{A}| - 1) \text{ if } a = \arg\max_a q_{t+1}(s_t, a)$$
$$\pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}|} \text{ otherwise}$$

expected sarsa，只改了 TD target，

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - (r_{t+1} + \gamma \mathbb{E}[q_t(s_{t+1}, A)])\right],$$
$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t),$$

where

$$\mathbb{E}[q_t(s_{t+1}, A)]) = \sum_a \pi_t(a|s_{t+1}) q_t(s_{t+1}, a) \doteq v_t(s_{t+1})$$

-

n-step sarsa，记住 action value 在 model free 的定义就是做完这一步后 reward 的条件期望！即下面的定义

- The definition of action value is

$$q_\pi(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a].$$

The discounted return G_t can be written in different forms as

$$\begin{aligned}
\text{Sarsa} \longleftarrow \quad & G_t^{(1)} = R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}), \\
& G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, A_{t+2}), \\
& \vdots \\
n\text{-step Sarsa} \longleftarrow \quad & G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n}), \\
& \vdots \\
\text{MC} \longleftarrow \quad & G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots
\end{aligned}$$

补充 | 一步奖励与累计回报：$q_\pi(s,a)$ 是先执行 a、随后遵循 π 的**累计回报**条件期望，并非只有一步奖励。固定策略的 TD 评价求解贝尔曼期望方程；Q-learning 才使用带最大化的最优目标。下面 V_t 是当前价值估计，Y_t 是一步 TD 目标；TD error 衡量两者差异，不能直接当作与真实价值之差。

$$q_\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a], \qquad Y_t = R_{t+1} + \gamma V_t(S_{t+1}), \qquad \delta_t = Y_t - V_t(S_t).$$

Q-learning：依旧是只改变 TD-target

- $$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\left[q_t(s_t,a_t) - [r_{t+1} + \gamma \max_{a\in\mathcal{A}} q_t(s_{t+1},a)]\right],$$
$$q_{t+1}(s,a) = q_t(s,a), \quad \forall (s,a) \neq (s_t,a_t),$$

Q-learning is very similar to Sarsa. They are different only in terms of the TD target:

- The TD target in Q-learning is $r_{t+1} + \gamma \max_{a\in\mathcal{A}} q_t(s_{t+1},a)$
- The TD target in Sarsa is $r_{t+1} + \gamma q_t(s_{t+1},a_{t+1})$.

on-policy and off-policy, 要判断首先要判断是在解决什么问题

- There exist two policies in a TD learning task:
  - The behavior policy is used to generate experience samples.
  - The target policy is constantly updated toward an optimal policy.
- On-policy vs off-policy:
  - When the behavior policy is the same as the target policy, such kind of learning is called on-policy.
  - When they are different, the learning is called off-policy.

Q learning 的 on-policy 和 off policy 版本，先有数据 experience 来计算 action value，然后再更新 policy，然后用新的 policy 再得到数据

- For each episode, do
  - If the current s_t is not the target state, do
    - Collect the experience $(s_t, a_t, r_{t+1}, s_{t+1})$: In particular, take action a_t following $\pi_t(s_t)$, generate r_{t+1}, s_{t+1}.
    - *Update q-value:*

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\Big[q_t(s_t, a_t) - [r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)]\Big]$$

    - *Update policy:*

$$\pi_{t+1}(a|s_t) = 1 - \frac{\epsilon}{|\mathcal{A}|}(|\mathcal{A}| - 1) \text{ if } a = \arg\max_a q_{t+1}(s_t, a)$$
$$\pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}|} \text{ otherwise}$$

π_b 和 π_T 不相同，off-policy

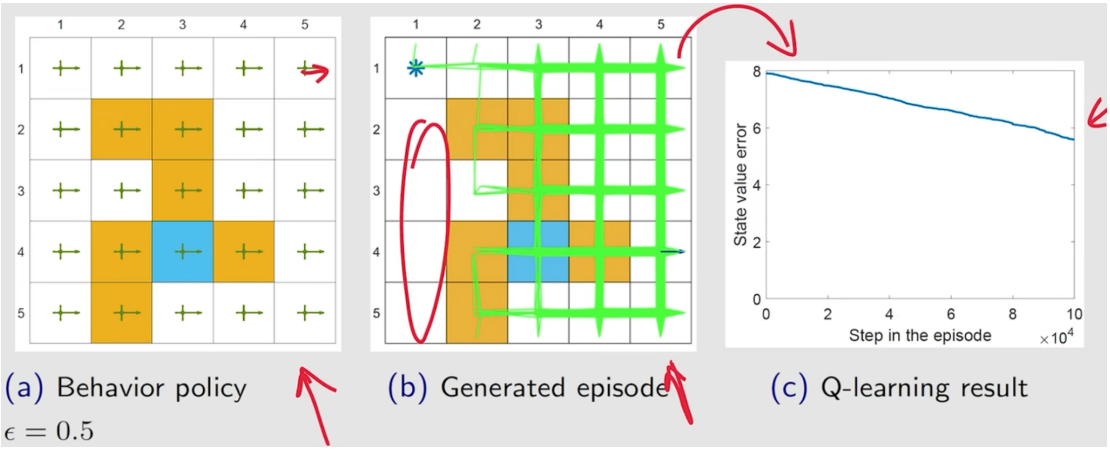
- For each episode $\{s_0, a_0, r_1, s_1, a_1, r_2, \ldots\}$ generated by π_b, do
  - For each step $t = 0, 1, 2, \ldots$ of the episode, do
    - *Update q-value:*

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\Big[q(s_t, a_t) - [r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)]\Big]$$

    - *Update target policy:*

$$\pi_{T,t+1}(a|s_t) = 1 \text{ if } a = \arg\max_a q_{t+1}(s_t, a)$$
$$\pi_{T,t+1}(a|s_t) = 0 \text{ otherwise}$$

理解这个 behavior policy，应该要探索性比较强

- 

(a) Behavior policy $\epsilon = 0.5$ (b) Generated episode (c) Q-learning result

all in all

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)[q_t(s_t, a_t) - \bar{q}_t],$$

where $\bar{q}_t$ is the *TD target*.

Different TD algorithms have different $\bar{q}_t$.

-

Algorithm	Expression of $\bar{q}_t$
Sarsa	$\bar{q}_t = r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$
n-step Sarsa	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})$
Expected Sarsa	$\bar{q}_t = r_{t+1} + \gamma \sum_a \pi_t(a\|s_{t+1}) q_t(s_{t+1}, a)$
Q-learning	$\bar{q}_t = r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$
Monte Carlo	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \ldots$

recap bellman equation，求的是一个策略的价值 action value/state value（在状态 s 先做动作 a，之后遵循策略 π 的预期累计回报或从状态 s 开始，遵循策略 π 的预期累计回报），bellman optimal equation 是通过 action value 选最好的 policy

-

Algorithm	Equation aimed to solve
Sarsa	BE: $q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \| S_t = s, A_t = a]$
n-step Sarsa	BE: $q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(s_{t+n}, a_{t+n}) \| S_t = s, A_t = a]$
Expected Sarsa	BE: $q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \mathbb{E}_{A_{t+1}}[q_\pi(S_{t+1}, A_{t+1})] \middle\| S_t = s, A_t = a\right]$
Q-learning	BOE: $q(s,a) = \mathbb{E}\left[R_{t+1} + \max_a q(S_{t+1}, a) \middle\| S_t = s, A_t = a\right]$
Monte Carlo	BE: $q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \ldots \| S_t = s, A_t = a]$

lec8 值函数近似 (value function approximation)

1. Motivating examples: curve fitting ←
2. Algorithm for state value estimation
   - Objective function
   - Optimization algorithms
   - Selection of function approximators
   - Illustrative examples
   - Summary of the story
   - Theoretical analysis
3. Sarsa with function approximation
4. Q-learning with function approximation
5. Deep Q-learning
6. Summary

- motivating example：$V_\pi(s)$ 和 $q_\pi(s,a)$ 之前都是用网格方式来存储，对于连续空间或很大的时候对于存储和泛化能力都比较棘手。

$$\hat{v}(s,w) = as + b = \begin{bmatrix} s & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \phi^T(s)w$$

- where:
  - w is the **parameter vector**.
  - $\phi(s)$ is the **feature vector** of s .
  - $\hat{v}(s,w)$ is **linear in** w .
- 其中：

$$\phi(s) = [s,1]^T, \quad w = [a,b]^T$$

对于函数近似的 goal，用函数近似的好熟是泛化能力，在更新了一个位置之后 w 权重会更新，所以曲线整体也会更新

- Our goal is to find an optimal w so that $\hat{v}(s,w)$ can best approximate $v_\pi(s)$ for every s.
- 目标函数 objective function：MSE

stationary distribution weighted？ squared error

- Stationary distribution is an important concept that will be frequently used in this course. In short, it describes the long-run behavior of a Markov process.
- Let $\{d_\pi(s)\}_{s\in\mathcal{S}}$ denote the stationary distribution of the Markov process under policy π. By definition, $d_\pi(s) \geq 0$ and $\sum_{s\in\mathcal{S}} d_\pi(s) = 1$.
- The objective function can be rewritten as

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \sum_{s\in\mathcal{S}} d_\pi(s)(v_\pi(s) - \hat{v}(s,w))^2.$$

This function is a weighted squared error.

- 理解：$d_\pi(s)$ 就是在一个 epsilon greedy 情况下跑了非常大的 episode 中到某个点的次数除以所有点的次数
P_π 是状态转移矩阵，从 s 到 s' 概率

The converged values can be predicted because they are the entries of d_π:

$$d_\pi^T = d_\pi^T P_\pi$$

For this example, we have P_π as

$$P_\pi = \begin{bmatrix} 0.3 & 0.1 & 0.6 & 0 \\ 0.1 & 0.3 & 0 & 0.6 \\ 0.1 & 0 & 0.3 & 0.6 \\ 0 & 0.1 & 0.1 & 0.8 \end{bmatrix}.$$

It can be calculated that the left eigenvector for the eigenvalue of one is

$$d_\pi = \begin{bmatrix} 0.0345, 0.1084, 0.1330, 0.7241 \end{bmatrix}^T$$

gradient desent 求 W，$v_\pi(s)$ 未知因为 state value 是期望，蒙德卡洛就用样本近似期望，这个也叫做 linear function approximation，因为对于 W 是线性的

- $$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k)$$

  The true gradient is

  $$\begin{aligned}\nabla_w J(w) &= \nabla_w \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] \\ &= \mathbb{E}[\nabla_w (v_\pi(S) - \hat{v}(S,w))^2] \\ &= 2\mathbb{E}[(v_\pi(S) - \hat{v}(S,w))(-\nabla_w \hat{v}(S,w))] \\ &= -2\mathbb{E}[(v_\pi(S) - \hat{v}(S,w))\nabla_w \hat{v}(S,w)]\end{aligned}$$

  - First, **Monte Carlo learning with function approximation**
    Let g_t be the discounted return starting from s_t in the episode. Then, g_t can be used to approximate $v_\pi(s_t)$. The algorithm becomes

    $$w_{t+1} = w_t + \alpha_t(g_t - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t).$$

  - Second, **TD learning with function approximation**
    By the spirit of TD learning, $r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t)$ can be viewed as an approximation of $v_\pi(s_t)$. Then, the algorithm becomes

    $$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t).$$

- value function approximation
  deep Q learning（network）DQN the loss function is actually the bellman optimality error, w 是学习参数，即神经网络权重，DQN 是去学习这个权重去学习输入 s, a , 输出 $\hat{q}_\pi(s,a)$ 的映射

- Deep Q-learning aims to minimize the objective function/loss function:

  $$J(w) = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right],$$

  where (S, A, R, S') are random variables.

  - This is actually the Bellman optimality error. That is because

    $$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a \in \mathcal{A}(S_{t+1})} q(S_{t+1}, a) \middle| S_t = s, A_t = a\right], \quad \forall s, a$$

再更新 w 的时候 w_T 保持不动，视为常数，最初这两个是一样的

- One is a main network representing $\hat{q}(s,a,w)$
- The other is a target network $\hat{q}(s,a,w_T)$.

The objective function in this case degenerates to

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S',a,w_T) - \hat{q}(S,A,w)\right)^2\right],$$

When w_T is fixed, the gradient of J can be easily obtained as

$$\nabla_w J = \mathbb{E}\left[\left(R + \gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S',a,w_T) - \hat{q}(S,A,w)\right)\nabla_w \hat{q}(S,A,w)\right]$$

经验回放 experience replay，tabular Q learning 不需要因为可以直接从表格中取 Q 函数值，DQN 需要时因为现在 q 函数是 $Q(S,A,w)$，看上文，是一个期望函数，所以要求分布，以及均匀采样的定义是缓冲区中每条经验被抽中的概率相同

However, the samples are not uniformly collected because they are generated consequently by certain policies.

To break the correlation between consequent samples, we can use the experience replay technique by uniformly drawing samples from the replay buffer.

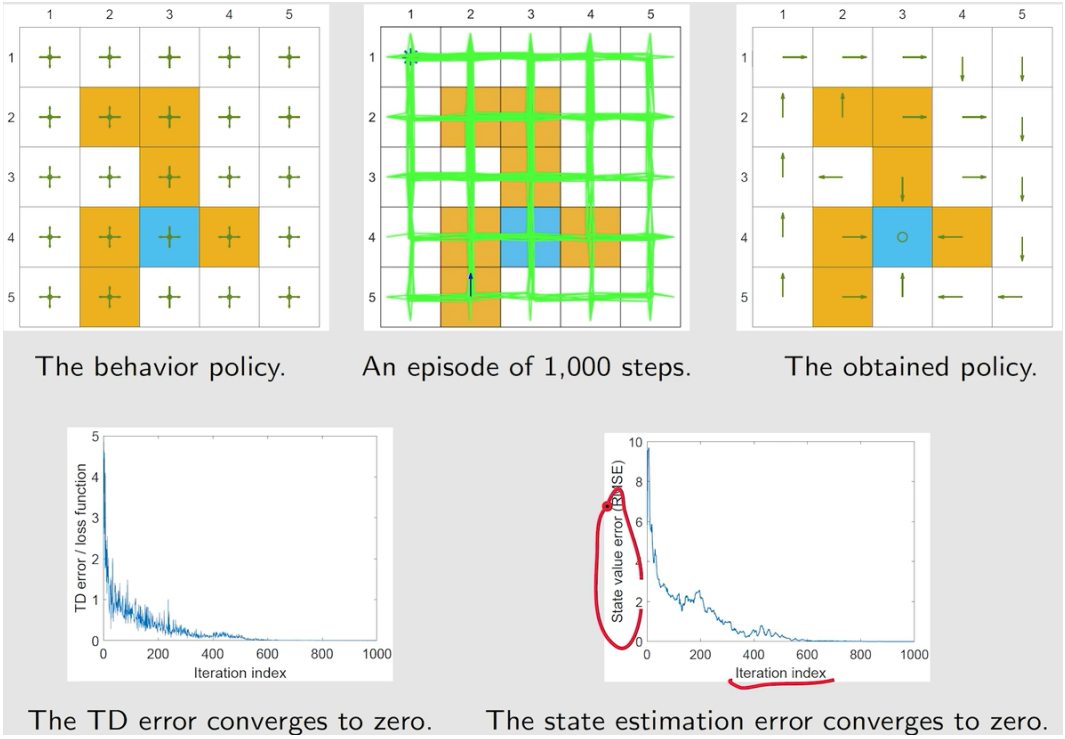
This is the mathematical reason *why experience replay is necessary* and *why the experience replay must be uniform*.

补充 | 为什么回放：相邻交互样本通常相关，同一网络的更新又会影响许多状态的估计。经验回放通过随机抽取、重复利用旧经验来减弱这种相关性并提高数据利用率；目标网络暂时固定更新目标。这两者作用不同，表格方法也可以使用回放。

DQN 伪代码，理解 3 点，1. 神经网络是关于 W 权重的，输入 s, a , 输出 $\hat{q}$ 2.target value 是 y_T ，希望 $\hat{q}$ 接近 y_T

- Store the experience samples generated by π_b in a replay buffer $\mathcal{B} = \{(s, a, r, s')\}$
  For each iteration, do
  Uniformly draw a mini-batch of samples from $\mathcal{B}$
  For each sample (s, a, r, s'), calculate the target value as $y_T = r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$, where w_T is the parameter of the target network
  Update the main network to minimize $(y_T - \hat{q}(s, a, w))^2$ using the mini-batch $\{(s, a, y_T)\}$
  Set $w_T = w$ every C iterations

deep q learning 例子

- 

lec 9 policy gredient （policy function approximation)

- 直接用 metric 定义什么样的 policy 是最优的，now policy can be represented by parameterized functions: $\pi(a \mid s, \theta)$, θ 是参数向量, 这个是 policy 的表示，我们要最大化的是指标
  图表和函数的区别：1. 最优 policy 定义的区别 2.how to access the possibility of an action 前向传播 3.how to update policies 反向传播

- When represented as a table, a policy π is optimal if it can maximize *every state value*.
  When represented by a function, a policy π is optimal if it can maximize certain *scalar metrics*.

- metric 指标？1.average value
- 对于 $J(\theta) = d^\top V_\pi$ 1. d 与 policy 无关，则求梯度的时候不包含 2. d 与 policy 有关，则求梯度还要对 d 求导，这个 J 实际上就是评价这个策略在不同状态下的整体表现，因为 state value 是在某个特定状态下策略的表现，这也解释了为什么我们要用 stationary distribution。
  通常我们可以用 stationary distribution，而且可以用 P 矩阵直接算出来
- A common way to select d as $d_\pi(s)$, which is the stationary distribution under π. Details of stationary distribution can be found in the last lecture and the book.
  - One basic property of d_π is that it satisfies

$$d_\pi^T P_\pi = d_\pi^T,$$

  where P_π is the state transition probability matrix.

$\bar{v}_\pi$ 也可以这样写

- $$\begin{aligned} J(\theta) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right] &= \sum_{s\in\mathcal{S}} d(s)\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} | S_0 = s\right] \\ &= \sum_{s\in\mathcal{S}} d(s) v_\pi(s) \\ &= \bar{v}_\pi \end{aligned}$$

the second metric 指标 $\bar{r}_\pi$ 和 $r_\pi(s)$ 有关，其中 $r_\pi(s)$ 在策略下走一步的 reward 下图相当于求了两次期望（加权平均），$\bar{r}_\pi$ 实际上是统计策略长期运行时，每一步平均能获得多少奖励。

- In particular, the metric is

$$\bar{r}_\pi \doteq \sum_{s\in\mathcal{S}} d_\pi(s) r_\pi(s) = \mathbb{E}[r_\pi(S)],$$

$\sim d_\pi$. Here,

$$r_\pi(s) \doteq \sum_{a\in\mathcal{A}} \pi(a|s) r(s,a)$$

erage of the one-step immediate reward that can

rom state s, and

$$r(s,a) = \mathbb{E}[R|s,a] = \sum_r r p(r|s,a)$$

reward 还有另外一种计算式

- $$\lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n} R_{t+k}|S_t = s_0\right] = \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n} R_{t+k}\right]$$
$$= \sum_s d_\pi(s) r_\pi(s)$$
$$= \bar{r}_\pi$$

$\bar{r}_\pi$ 和 $\bar{v}_\pi$ 实际上是可以用一个等式连接的，其中一个达到极大的时候另一个也达到极大

- However, the two metrics are equivalent to each other. In the discounted case where $\gamma < 1$, it holds that

$$\bar{r}_\pi = (1-\gamma)\bar{v}_\pi.$$

下面我门来求梯度，实际上求梯度是非常复杂的，可以自行探索，下面直接给出，这个 J 可以是 $\bar{v}_\pi$ 也可以是 $\bar{r}_\pi$

- $$\nabla_\theta J(\theta) = \sum_{s\in\mathcal{S}} \eta(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a|s,\theta) q_\pi(s,a)$$

一个重要的转化以及证明，回忆对 $\ln(x)$ 求导

- **and useful form of the gradient:**

$$\begin{aligned}\nabla_\theta J(\theta) &= \sum_{s\in\mathcal{S}} \eta(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a|s,\theta) q_\pi(s,a) \\ &= \mathbb{E}\left[\nabla_\theta \ln \pi(A|S,\theta) q_\pi(S,A)\right]\end{aligned}$$

and $A \sim \pi(A|S,\theta)$.

$$\nabla_\theta \ln \pi(a|s,\theta) = \frac{\nabla_\theta \pi(a|s,\theta)}{\pi(a|s,\theta)}$$

$$\begin{aligned}&= \sum_s d(s) \sum_a \pi(a|s,\theta) \nabla_\theta \ln \pi(a|s,\theta) q_\pi(s,a) \\ &= \mathbb{E}_{S\sim d}\left[\sum_a \pi(a|S,\theta) \nabla_\theta \ln \pi(a|S,\theta) q_\pi(S,a)\right] \\ &= \mathbb{E}_{S\sim d, A\sim\pi}\left[\nabla_\theta \ln \pi(A|S,\theta) q_\pi(S,A)\right] \\ &\doteq \mathbb{E}\left[\nabla_\theta \ln \pi(A|S,\theta) q_\pi(S,A)\right]\end{aligned}$$

- softmax functions，用 softmax 归一化让 $\ln(x)$ 中的 x 始终大于 0

将梯度转变为真正能计算的梯度下降，要经历两个过程。1. 把这个 expectation 变成随机抽样。2. 把这个 q_π 表示的真实 action value 变成近似

- $$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \nabla_\theta J(\theta) \\ &= \theta_t + \alpha \mathbb{E}\Big[\nabla_\theta \ln \pi(A|S,\theta_t) q_\pi(S,A)\Big]\end{aligned}$$

and this is on-policy

- $$\mathbb{E}_{S\sim d, A\sim\pi}\Big[\nabla_\theta \ln \pi(A|S,\theta_t) q_\pi(S,A)\Big] \longrightarrow \nabla_\theta \ln \pi(a|s,\theta_t) q_\pi(s,a)$$
  - How to sample S?
    - $S \sim d$, where the distribution d is a long-run behavior under π.
  - How to sample A?
    - $A \sim \pi(A|S,\theta)$. Hence, a_t should be sampled following $\pi(\theta_t)$ at s_t.
    - Therefore, the policy gradient method is on-policy.

- $\pi(a_t \mid s_t, \theta)$ 是指的是在状态 s_t 下选择动作 a_t 的概率，β_t , expolitation, exploration
  伪代码，先生成出一些序列，注意蒙德卡洛是 offline 的，更新的函数是 $\pi(a_t \mid s_t, \theta)$ 而不是 action value，这个 policy update 是 stochastic gradient-ascent

Initialization: A parameterized function $\pi(a|s,\theta)$, $\gamma \in (0,1)$, and $\alpha > 0$.

Aim: Search for an optimal policy maximizing $J(\theta)$.

- For the kth iteration, do
  - Select s_0 and generate an episode following $\pi(\theta_k)$. Suppose the episode is $\{s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T\}$.
  - For $t = 0, 1, \ldots, T-1$, do
    - Value update: $q_t(s_t, a_t) = \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$
    - Policy update: $\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t) q_t(s_t,a_t)$
  - $\theta_k = \theta_T$

lec10 actor critic

1. The simplest actor-critic (QAC)
2. Advantage actor-critic (A2C)
   - Baseline invariance
   - The algorithm of advantage actor-critic
3. Off-policy actor-critic
   - Illustrative examples
   - Importance sampling
   - The theorem of off-policy policy gradient
   - The algorithm of off-policy actor-critic
4. Deterministic actor-critic (DPG)
   - The theorem of deterministic policy gradient
   - The algorithm of deterministic actor-critic

- actor critic methods are still policy gradient methods.
- actor refers to policy update; critic refers to policy evaluation/value estimation
  actor 是式子 3），critic 是如何计算 $q_t(s_t, a_t)$ ， 下面

- 2) The gradient-ascent algorithm maximizing $J(\theta)$ is

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \nabla_\theta J(\theta_t) \\ &= \theta_t + \alpha \mathbb{E}_{S\sim\eta, A\sim\pi}\left[\nabla_\theta \ln \pi(A|S,\theta_t) q_\pi(S,A)\right]\end{aligned}$$

3) The stochastic gradient-ascent algorithm is

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t) q_t(s_t,a_t)$$

critic 像是 sarsa+value function approximation；actor correspond to policy update algorithm，QAC

Aim: Search for an optimal policy by maximizing $J(\theta)$.

At time step t in each episode, do

Generate a_t following $\pi(a|s_t,\theta_t)$, observe r_{t+1}, s_{t+1}, and then generate a_{t+1} following $\pi(a|s_{t+1},\theta_t)$.

Critic (value update):

$$w_{t+1} = w_t + \alpha_w\big[r_{t+1} + \gamma q(s_{t+1},a_{t+1},w_t) - q(s_t,a_t,w_t)\big]\nabla_w q(s_t,a_t,w_t)$$

Actor (policy update):

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \ln \pi(a_t|s_t,\theta_t) q(s_t,a_t,w_{t+1})$$

Baseline（偏置）invariance valid&useful

- $$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\eta, A\sim\pi}\Big[\nabla_\theta \ln \pi(A|S,\theta_t) q_\pi(S,A)\Big]$$
$$= \mathbb{E}_{S\sim\eta, A\sim\pi}\Big[\nabla_\theta \ln \pi(A|S,\theta_t)(q_\pi(S,A) - b(S))\Big]$$

求 optimal baseline 是为了使得 $E[X]$ 的方差最小，这里蓝色是先求梯度再求模再求平方，实际用 state value

- The optimal baseline that can minimize $\mathrm{var}(X)$ is, for any $s \in \mathcal{S}$,

$$b^*(s) = \frac{\mathbb{E}_{A\sim\pi}[\|\nabla_\theta \ln \pi(A|s,\theta_t)\|^2 q(s,A)]}{\mathbb{E}_{A\sim\pi}[\|\nabla_\theta \ln \pi(A|s,\theta_t)\|^2]}.$$

We can remove the weight $\|\nabla_\theta \ln \pi(A|s,\theta_t)\|^2$ and select the suboptimal baseline:

$$b(s) = \mathbb{E}_{A\sim\pi}[q(s,A)] = v_\pi(s)$$

A2C: Advantage function $\delta_\pi(S,A) = q_\pi(S,A) - v_\pi(S)$, 其中 v_π 可以看作是某一点在策略下所有可能的 action 的价值的平期望，所以如果 $q_\pi > v_\pi$, 说明在这个点这个 action 不错

$v_\pi(s)$	在状态 s 开始，遵循策略 π 的未来回报期望
$q_\pi(s,a)$	在状态 s 先执行动作 a，之后遵循策略 π 的未来回报期望

两者的关系是：

$$\boxed{v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s,a)}$$

- 也就是说，**状态价值等于各个动作价值按照策略选择概率加权后的平均值。**

gradient-ascent algorithm is

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \mathbb{E}\Big[\nabla_\theta \ln \pi(A|S,\theta_t)[q_\pi(S,A) - v_\pi(S)]\Big] \\ &\doteq \theta_t + \alpha \mathbb{E}\Big[\nabla_\theta \ln \pi(A|S,\theta_t)\delta_\pi(S,A)\Big]\end{aligned}$$

stochastic version of this algorithm is

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t)[q_t(s_t,a_t) - v_t(s_t)] \\ &= \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t)\delta_t(s_t,a_t)\end{aligned}$$

再把这个 advantage function 换成 TD error：$\delta_\pi = R_{t+1} + \gamma V_\pi(s_{t+1}) - v_\pi(s_t)$ 这样就可以少拟合一个 q_π 网络了

Aim: Search for an optimal policy by maximizing $J(\theta)$.

At time step t in each episode, do

Generate a_t following $\pi(a|s_t, \theta_t)$ and then observe r_{t+1}, s_{t+1}.

TD error (advantage function):

$$\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$$

Critic (value update):

$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w v(s_t, w_t)$$

Actor (policy update):

$$\theta_{t+1} = \theta_t + \alpha_\theta \delta_t \nabla_\theta \ln \pi(a_t|s_t, \theta_t)$$

补充 | 优势与 TD error：优势常记为 $A_\pi(s,a) = q_\pi(s,a) - v_\pi(s)$，是“该动作比策略平均水平好多少”；$\delta_t$ 是一次采样得到的 TD error。使用真实 v_π 、且后续遵循 π 时，有 $\mathbb{E}[\delta_t \mid S_t = s, A_t = a] = A_\pi(s,a)$ 。因此它是优势的采样估计，二者并非逐样本相等。

importance sampling：converting on-policy to off policy，$p_0(x_i)/p_1(x_i)$ is called the importance weight

$$\mathbb{E}_{X\sim p_0}[X] = \sum_x p_0(x)x = \sum_x p_1(x)\underbrace{\frac{p_0(x)}{p_1(x)}}_{f(x)}x = \mathbb{E}_{X\sim p_1}[f(X)]$$

有分布概率函数为什么不直接求期望?：

You may ask: While $\bar{f} = \frac{1}{n}\sum_{i=1}^n \frac{p_0(x_i)}{p_1(x_i)}x_i$ requires $p_0(x)$, if I know $p_0(x)$, why not directly calculate the expectation?

Answer: It is applicable to the case where it is easy to calculate $p_0(x)$ given an x, but difficult to calculate the expectation.

- For example, continuous case, complex expression of p_0, or no expression of p_0 (e.g., p_0 represented by a neural network).

off policy actor critic, 为什么这里 A 属于 β 就是 off policy 了?：之前 $A \sim \pi$ 代表在状态 S 下，动作 A 是按照我们正在优化的策略 π 采样的。现在，A 属于 β 代表我们收集到的数据中的动作，是由行为策略（β ）产生的, 而我们要优化的仍然是 π ，这样就分离了。同时，上面的 $\pi(A \mid S, \theta)$ 实际上对应 p_0 , $\beta(A \mid S)$ 对应 p_1 , 也就用到了 importance sampling。

- **Theorem (Off-policy policy gradient theorem)**

  In the discounted case where $\gamma \in (0,1)$, the gradient of $J(\theta)$ is

  $$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\rho, A\sim\beta}\left[\frac{\pi(A|S,\theta)}{\beta(A|S)}\nabla_\theta \ln \pi(A|S,\theta) q_\pi(S,A)\right]$$

  where β is the behavior policy and ρ is a state distribution.

  See the details and the proof in my book.

然后再加入 baseline 和 stochastic 方法去计算

- The corresponding stochastic gradient-ascent algorithm is

  $$\theta_{t+1} = \theta_t + \alpha_\theta \frac{\pi(a_t|s_t,\theta_t)}{\beta(a_t|s_t)}\nabla_\theta \ln \pi(a_t|s_t,\theta_t)\big(q_t(s_t,a_t) - v_t(s_t)\big)$$

  Similar to the on-policy case,

  $$q_t(s_t,a_t) - v_t(s_t) \approx r_{t+1} + \gamma v_t(s_{t+1}) - v_t(s_t) \doteq \delta_t(s_t,a_t)$$

  Then, the algorithm becomes

  $$\theta_{t+1} = \theta_t + \alpha_\theta \frac{\pi(a_t|s_t,\theta_t)}{\beta(a_t|s_t)}\nabla_\theta \ln \pi(a_t|s_t,\theta_t)\delta_t(s_t,a_t)$$

  and hence

  $$\theta_{t+1} = \theta_t + \alpha_\theta \left(\frac{\delta_t(s_t,a_t)}{\beta(a_t|s_t)}\right)\nabla_\theta \pi(a_t|s_t,\theta_t)$$

Initialization: A given behavior policy $\beta(a|s)$. A target policy $\pi(a|s,\theta_0)$ where θ_0 is the initial parameter vector. A value function $v(s,w_0)$ where w_0 is the initial parameter vector.

Aim: Search for an optimal policy by maximizing $J(\theta)$.

A2

At time step t in each episode, do

Generate a_t following $\beta(s_t)$ and then observe r_{t+1}, s_{t+1}.

TD error (advantage function):

$$\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$$

Critic (value update):

$$w_{t+1} = w_t + \alpha_w \frac{\pi(a_t|s_t,\theta_t)}{\beta(a_t|s_t)} \delta_t \nabla_w v(s_t, w_t)$$

Actor (policy update):

$$\theta_{t+1} = \theta_t + \alpha_\theta \frac{\pi(a_t|s_t,\theta_t)}{\beta(a_t|s_t)} \delta_t \nabla_\theta \ln \pi(a_t|s_t,\theta_t)$$

- deterministic actor critic

  stochastic 和 deteministic 相当于是两个方向？stochastic 表达的是输入 s，然后输入对应的 action 概率，对应的概率全部都大于 0 且相加等于 1，那么这里的 action 必须是有限的，如果在一个状态 s 有无限个 action，所以用 deterministic 来 handle continuous action

  - Up to now, a general policy is denoted as $\pi(a|s,\theta) \in [0,1]$, which can be either stochastic or deterministic.
  - Now, the deterministic policy is specifically denoted as

    $$a = \mu(s,\theta) \doteq \mu(s)$$

    $\mu : \mathcal{S} \to \mathcal{A}$

    - μ is a mapping from $\mathcal{S}$ to $\mathcal{A}$.
    - μ can be represented by, for example, a neural network with the input as s, the output as a, and the parameter as θ.
    - We may write $\mu(s,\theta)$ in short as $\mu(s)$.

  补充 | 连续动作也能随机：随机策略可以输出连续动作的概率密度，例如高斯分布的均值、标准差，再从中采样；概率密度通过积分归一化。确定性策略直接输出一个动作。是否 off-policy 取决于采集数据的行为策略与学习的目标策略是否一致，不能仅由“确定性”判断。

理解这个 theorem: $a = \mu(s;\theta)$, 是权重为 θ 的神经网络中输入 s_t 输出确定 a_t ， $\rho_\mu(s)$ 是机器人执行策略 μ 时，哪些状态更常被访问

- **Theorem (Deterministic policy gradient theorem in the discounted case)**

  In the discounted case where $\gamma \in (0,1)$, the gradient of $J(\theta)$ is

  $$\nabla_\theta J(\theta) = \sum_{s\in\mathcal{S}} \rho_\mu(s) \nabla_\theta \mu(s) \left(\nabla_a q_\mu(s,a)\right)|_{a=\mu(s)}$$
  $$= \mathbb{E}_{S\sim\rho_\mu}\left[\nabla_\theta \mu(S) \left(\nabla_a q_\mu(S,a)\right)|_{a=\mu(S)}\right]$$

  Here, ρ_μ is a state distribution.

  See more details and the proof in my book.

$$\underbrace{\text{参数如何改变动作}}_{\nabla_\theta \mu(s)} \times \underbrace{\text{动作如何改变价值}}_{\nabla_a q_\mu(s,a)} \implies \underbrace{\text{参数如何改变性能}}_{\nabla_\theta J(\theta)}$$

- 这个梯度没有涉及到 action A 的 distribution，所以，这个 deterministic 是 off policy 的，因为在采集动作序列的时候不关心这个 a 是由哪个 policy 得到的，最本质的还是 β 与 μ 是不同的策略，当然可以加入随机噪声去让它更有探索性。
  DPG 指的是 deterministisitic policy gradient，DDPG 就是 deep deterministic policy gradient，主要是看用哪个神经网络去拟合 $q(s,a;\theta)$, 当然还有 $\mu(s;\theta)$

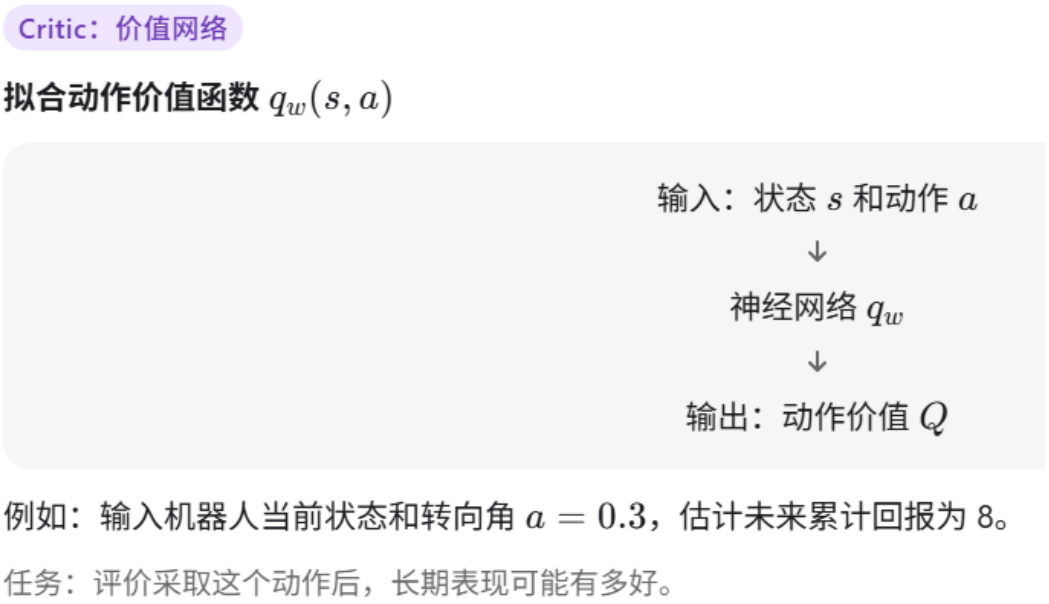
- Actor：策略网络

  拟合确定性策略 $\mu_\theta(s)$

  输入：状态 s
  ↓
  神经网络 μ_θ
  ↓
  输出：动作 a

  例如：输入机器人的位置、速度，输出转向角 $a = 0.3$。

  任务：决定应该采取什么动作。



补充说明参考：Sutton 与 Barto《Reinforcement Learning: An Introduction》第二版，第 3、4、6、11、13 章。原笔记表述保留，新增解释均以 “补充” 标记。